

## 5.8     *Osnove objektnog programiranja u C++*

- 5.8.1    Klase i objekti
- 5.8.2    Članovi klasa
- 5.8.3    Adresiranje objekata klasa
- 5.8.4    Konstruktor i destruktor
- 5.8.5    Pokazivač this
- 5.8.6    Specifikatori pristupa
- 5.8.7    Nasljeđivanje
- 5.8.8    Virtualni funkcijski članovi i polimorfizam
- 5.8.9    Apstraktne klase i čiste virtualne funkcije

## 5.8 Osnove objektnog programiranja u C++

### 5.8.1 Klase i objekti

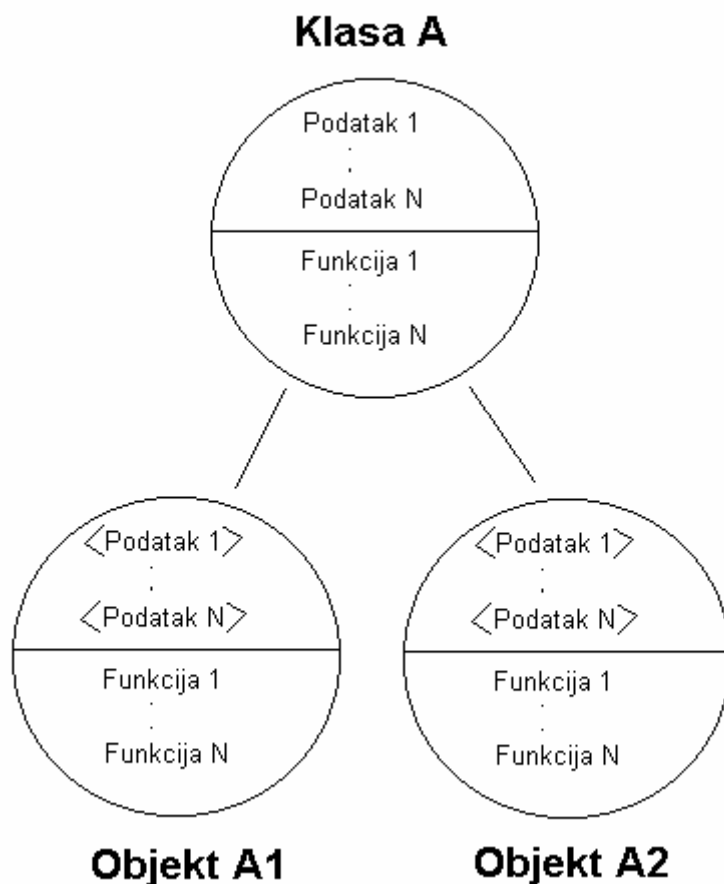
Klase su temelj objektno orijentiranog pristupa programiranju. Objekt je naziv za skup svojstava koja možemo objediniti u smislenu cjelinu, a klase su pravila koja propisuju od čega je pojedini objekt sagrađen te kakva su njegova svojstva. Klasa (razred) je samo opis, a objekt je stvarna, konkretna realizacija tog opisa.

U programskom jeziku C++ klase predstavljaju novinu u odnosu na klasični C. U osnovi, klasa u C++ je proširena struktura (ključna riječ *struct*) koja osim podataka (tzv. *podatkovnih članova*) sadrži i funkcije (tzv. *funkcijske članove*) koje određuju ponašanje objekata. Objekti predstavljaju klase inicijalizirane u memoriji.

#### 5.8.1.1 Definiranje klase i pripadajućih objekata

Definicija klase razlikuje se od definicija struktura u slijedećem :

- klasa obično sadrži specifikatore pristupa koji su definirani ključnim riječima *public*, *protected* i *private*,
- umjesto ključne riječi *struct* može se koristiti ključna riječ *class* ili *union*.
- klasa može sadržavati i funkcijske članove, osim podatkovnih i
- klasa može sadržavati i specijalne funkcijske članove, kao što su konstruktor (inicijalizira objekt) i destruktor (dealocira objekt).



Slika 7.1 : Klasa i objekti

U sintaksi jezika C++ klasa predstavlja tip podataka, a implementacije tog tipa su objekti.

#### Primjer (klasa):

```
class A {  
    public:  
        int podatak1;  
        char podatak2;  
        float podatak3;  
  
        void funkcija1();    // navodi se deklaracija (zaglavlje) svake funkcije,  
        int funkcija2();    // dok definicija (tijelo funkcije) može (ali ne mora)  
        float funkcija3();  // biti izdvojena  
};  
  
A A1,A2;                    // objekti A1 i A2 su iz klase A
```

### 5.8.2 Članovi klase

Klase definiraju objekte, a objekti sadrže konkretne podatke i postupke (metode) nad tim podacima. Podaci su definirani tzv. *podatkovnim članovima klase*, a postupci tzv. *funkcijskim članovima klase*.

#### Primjer (članovi klase):

```
class objekt{  
    public:  
        int pod1;            // podatkovni članovi  
        float pod2;  
  
        void f_prva (){      // umetnuti funkcijski član klase  
            .  
            .  
            .  
        };  
        int f_druga ();      // funkcijski član klase - deklaracija  
};  
int objekt::f_druga(){       // funkcijski član klase - definicija  
    .  
    .  
    .  
};
```

Funkcijski članovi klase mogu biti umetnuti, ako se njihova definicija (programski kod) nalazi unutar definicije klase. U suprotnom, definicija klase sadrži samo deklaraciju funkcijskog člana, dok se definicija nalazi negdje drugdje u programskom kodu. Razlika između “običnih” funkcija i funkcijskih članova klase je u području doseg: funkcije se nalaze unutar globalnog područja, dok su funkcijski članovi lokalni za klasu gdje su definirani, a dostupnost im je (kao i podatkovnim članovima klase) određena tzv. specifikatorima pristupa.

### 5.8.3 Adresiranje objekata klase

Adresiranje objekata klase ne razlikuje se od adresiranja implementacija struktura, odnosno, koriste se isti *operatori za pristup objektima/članovima objekata*:

- Objekt se adresira pomoću naziva objekta, reference ili pokazivača na objekt.
- Način adresiranja članova objekata ovisi o načinu adresiranja objekta, odnosno:
  - ako je objekt referenciran pomoću naziva objekta ili reference, za pristup članovima koristi se operator . (točka)
  - ako je objekt referenciran pomoću pokazivača, tada se za pristup članovima koristi operator -> (minus i veće od).

#### Primjer (adresiranje objekata klasa):

```
class student{
public:
    int mbr; //matični broj
    char pi[30]; //prezime i ime
    int g_stu; //godina studija
    void ispis(){ //funkcijski član klase student (umetnuti funkcijski član)
        cout << "Student : "<< pi << " Matični broj : ";
        cout << mbr << " Godina studija :." << g_stu << endl;
    };
};

student prvi; // objekt prvi identificiran je svojim nazivom
student &drugi = prvi; // objekt drugi identificiran je referencom, time ujedno
// kažemo da su objekti prvi i drugi zapravo isti objekt (jer
// zauzimaju isti memorijski blok)
student *treci; // objekt je identificiran pomoću pokazivača treći
```

Pristup članovima objekata može se vidjeti iz slijedećeg primjera :

```
prvi.mbr = 43500;
strcpy (prvi.pi, "Matić Mario");
```

odnosno :

```
drugi.g_stu = 2;
drugi.ispis(); // poziv funkcijskog člana
```

U slučaju da koristimo objekt \*treci, pristup bi bio slijedeći :

```
treci = new student; // rezerviramo memorijski prostor za objekt treci
treci -> mbr = 44311;
strcpy (treci -> pi, "Perić Marija");
treci -> g_stu = 3;
treci -> ispis(); // poziv funkcijskog člana
```

### 5.8.4 Konstruktor i destruktor

Konstruktor je specijalan funkcijski član koji ima isti naziv kao i klasa, a poziva se automatski kod inicijalizacije objekta. U odnosu na ostale funkcijske članove, konstruktor ima slijedeće specifičnosti :

- konstruktor nema definiran tip (ne vraća vrijednost),
- konstruktor se ne nasljeđuje,
- može postojati više konstruktora za istu klasu (odnosno, može se preopteretiti),

- konstruktor ne može biti statički član.

Destruktor također ima naziv isti kao i klasa, samo što ispred naziva klase dolazi znak ~ (tilda). Poziva se automatski kod uklanjanja objekta iz memorije. U odnosu na ostale funkcijske članove, konstruktor ima slijedeće specifičnosti :

- destruktor nema parametre,
- destruktor (kao i konstruktor) nema tip, ne vraća vrijednost i ne nasljeđuje se.

### Primjer (konstruktor i destruktor):

```
#include <iostream.h>

class konstruktor{
public:
    int pod1;
    char pod2;
    konstruktor(){                // konstruktor (nema tipa)
        cout << "Sada djeluje konstruktor (bez parametara)!" << endl;
    };

    konstruktor(int p1,char p2){    // konstruktor (s parametrima)
        cout << "Sada djeluje konstruktor (s parametrima)!" << endl;
        pod1=p1;
        pod2=p2;
        cout << "Pod1=" << pod1 << endl;
        cout << "Pod2=" << pod2 << endl;
    };
    ~konstruktor(){                // destruktor (također nema tipa)
        cout << "A sada destruktor!" << endl;
    };
};

konstruktor *konst;

void main(){
    konst = new konstruktor;        // Prilikom kreiranja objekta u memoriji
                                    // poziva se konstruktor, a kod njegovog
    delete konst;                    // uklanjanja iz memorije destruktor
    konst = new konstruktor (3,'A');
    delete konst;
}
```

Ispisuje se slijedeće :

```
Sada djeluje konstruktor (bez parametara)!
A sada destruktor!
Sada djeluje konstruktor (s parametrima)!
Pod1=3
Pod2=A
A sada destruktor!
```

### 5.8.5 Pokazivač *this*

Svaki funkcijski član klase (osim ako je statička) može pristupiti objektu kojem pripada upotrebom pokazivača *this*. Taj pokazivač je tipa *\*klasa*, a prosljeđuje se prilikom poziva svakog funkcijskog člana klase kao skriveni parametar. U biti, pokazivač *this* omogućuje objektu da u toku izvođenja programa dođe do svoje vlastite adrese u memoriji.

U najvećem broju slučajeva, pokazivač *this* se koristi interno od strane programa prevodioca, no može mu se pristupiti i iz programa. U slučaju da mu se pristupa iz programa, svrha može biti postavljanje pokazivača na objekt, kao što je to slučaj kod dodavanja novog objekta u dvostruko vezanu listu.

#### Primjer (pokazivač *this*):

```
#include <iostream.h>
```

```
class cthis{  
public:
```

```
    cthis *prethodni, *slijedeci;
```

```
    long fclan1(){ // 1. slučaj: objekt vraća svoju adresu  
        cout << "Vracam adresu objekta!" << endl;  
        return (long) this;
```

```
};
```

```
    void fclan2(){ // 2. slučaj: ovdje je navođenje this suvišno  
        cout << "Pozivam fclan1!" << endl;  
        long adr = this -> fclan1();  
        cout << "Adresa : " << adr << endl;
```

```
};
```

```
    void fclan3(){ // 3. slučaj: potrebno je postaviti pokazivač na objekt  
        // npr. u dvostruko vezanoj listi  
        cout << "Dodajem novi clan liste!" << endl;  
        cthis *novi_clan_liste = new cthis;  
        novi_clan_liste -> prethodni = this;  
        this -> slijedeci = novi_clan_liste; // ovdje je this suvišan
```

```
};
```

```
};
```

```
cthis *cthis_objekt, *c2;
```

```
void main(){
```

```
    cthis_objekt = new cthis;  
    long adresa = cthis_objekt -> fclan1();  
    cout << "Adresa objekta : " << adresa;  
    cthis_objekt -> fclan2();  
    cthis_objekt -> fclan3();
```

```
}
```

### 5.8.6 Specifikatori pristupa

Specifikatori pristupa određuju pravo pristupa pojedinim podatkovnim i funkcijskim članovima klase. U tu svrhu koriste se slijedeće ključne riječi : *public* (javni pristup), *private* (privatni pristup) i *protected* (zaštićeni pristup).

- **Javni pristup** (public) određuje da se članovima koji imaju takav pristup može neograničeno pristupiti unutar i izvan klase. Članovi koji imaju javni pristup čine tzv. *javno sučelje klase* jer preko njih klasa (odnosno njene implementacije – objekti) komuniciraju s ostalim dijelovima programa.

- **Privatni pristup** (private) određuje da odgovarajućim članovima mogu pristupiti isključivo funkcijski članovi iste klase. Za sve ostale dijelove programa (s izuzetkom *klase prijatelja*) ti članovi su nedostupni.

- **Zaštićeni pristup** (protected) određuje da članovi klase koje nasljeđuju osnovnu klasu mogu pristupiti odgovarajućim članovima osnovne klase, ali su ti članovi nedostupni iz svih ostalih dijelova programa.

U slučaju da pravo pristupa nije eksplicitno navedeno, podrazumijeva se privatni pristup.

### Primjer (specifikatori pristupa):

```
#include <iostream.h>
```

```
class cspecp { // osnovna klasa
```

```
public:
```

```
    int pub1, pub2;
```

```
    void fpub(){
```

```
        cout << "Javni funkcijski clan!" << endl;};
```

```
private:
```

```
    int priv1, priv2;
```

```
    void fpriv(){
```

```
        cout << "Privatni funkcijski clan!" << endl;};
```

```
protected:
```

```
    int prot1, prot2;
```

```
    void fprot(){
```

```
        cout << "Zasticeni funkcijski clan!" << endl;};
```

```
public:
```

```
    void fispis1(){
```

```
        fpub();
```

```
        fpriv();
```

```
        fprot();
```

```
    };
```

```
};
```

```
class cizved : public cspecp { // izvedena klasa
```

```
public:
```

```
    void fispis(){
```

```
        pub1=pub2=5;
```

```
        fpub();          // za javne članove nema ograničenja pristupa
```

```
        // priv1=priv2=10;
```

```
        // fpriv();       // nije dopušteno pristupati privatnim članovima
```

```
        prot1=prot2=15;
```

```
        fprot(); // dopušteno je, jer članovi izvedenih klasa mogu pristupiti
```

```
                // zaštićenim članovima osnovnih klasa
```

```
    };
```

```
};
```

```
cspecp *specp;
```

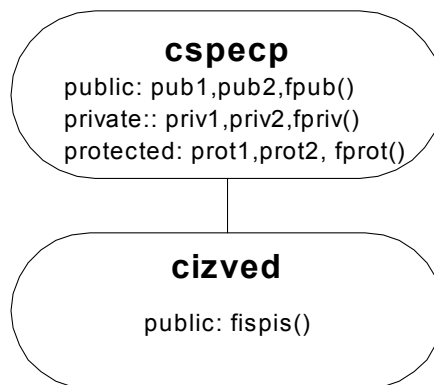
```
cizved *izved;
```

```

void main(){
    specp = new cspecp;
    izved = new cizved;
    cout << "Pristup članovima osnovne klase iz funkcije main()..." << endl;
    specp->pub1=specp->pub2=5;
    specp->fpub();           // za javne članove nema ograničenja pristupa
    // cspecp->priv1=cspecp->priv2=10;
    // cspecp->fpriv();      // nije dopušteno pristupati privatnim članovima

    // specp->prot1=specp->prot2=15;
    // specp->fprot();      // nije dopušteno pristupati zaštićenim
                          // članovima osnovnih klasa
                          // osim članovima osnovne i izvedenih klasa
    cout << "Pristup članovima osnovne klase iz funkcijskog člana izvedene
           klase..." << endl;
    izved->fispis();
    cout << "Pristup članovima osnovne klase iz funkcijskog člana osnovne
           klase..." << endl;
    specp->fispis1();
}

```



### 5.8.6.1 Prijatelj klase

Prijatelj klase predstavlja izuzetak od pravila da privatnim članovima klase mogu pristupati isključivo funkcijski članovi iste klase. Pojedina klasa može dodijeliti nekoj funkciji, funkcijskom članu neke klase ili čitavoj klasi pravo pristupa vlastitim privatnim i zaštićenim članovima. Takve funkcije, funkcijske članove i klase nazivamo *prijateljima klase*, a označavamo ih pomoću ključne riječi *friend*.

#### Primjer (prijatelj klase):

```
#include <iostream.h>
```

```

class ckolega{
public:
    void fclan1();
    void fclan2(); };
class cosnovna{

```

```

friend void funkcija1();           // funkcija1 je prijatelj klase cosnovna, a također i

```



```

friend void ckolega::fclan1();           // funkcijski član fclan1 klase ckolega,
friend class cprijatelj;                // kao i klasa cprijatelj

private:
    void privatna(){
        cout << "Privatni funkcijski član!" << endl;
    };
};

class cprijatelj{
public:
    void poziv(){
        cosnovna osnovna;
        osnovna.privatna();
    };
};

void funkcija1(){
    cosnovna osnovna;
    osnovna.privatna();
};

void ckolega::fclan1(){
    cosnovna osnovna;
    osnovna.privatna();
};

void ckolega::fclan2(){
    cosnovna osnovna;
    // osnovna.privatna();           // nije dopušteno jer fclan1 nije prijatelj
// klase cosnovna
};

cprijatelj prijatelj;
ckolega kolega;

void main(){
    cout << "Poziv iz 'prijateljske' funkcije:" << endl;
    funkcija1();
    cout << "Poziv iz objekta 'prijateljske' klase:" << endl;
    prijatelj.poziv();
    cout << "Poziv iz 'prijateljskog' funkcijskog člana:" << endl;
    kolega.fclan1();
}

```

U primjeru klasa *cosnovna* dopušta pristup svom privatnom funkcijskom članu:

- vanjskoj funkciji (funkcija1),
- svim članovima druge klase (cprijatelj) i
- funkcijskom članu druge klase (fclan1 iz klase ckolega).

Za deklaracije prijatelja klase vrijede slijedeća pravila:

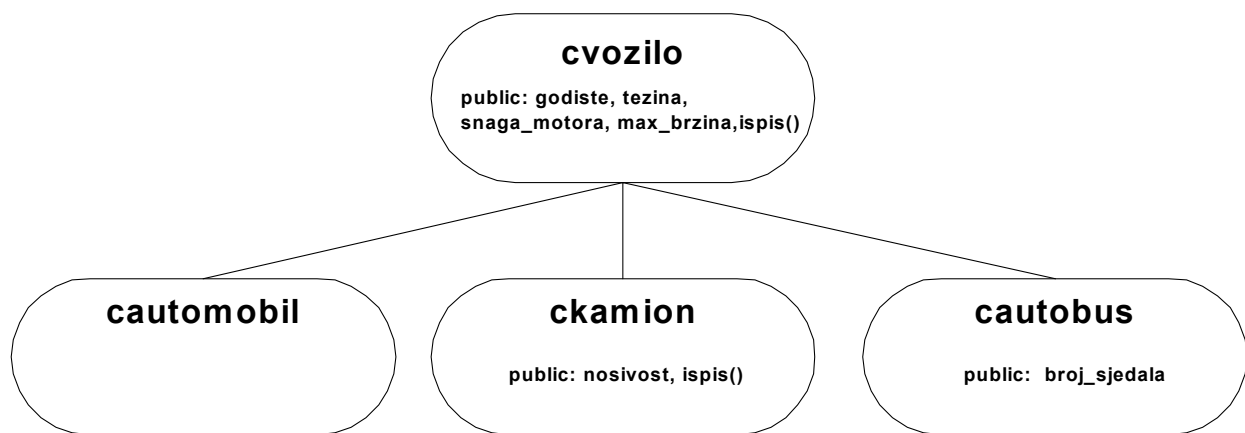
- Specifikatori pristupa (public, private i protected) nemaju utjecaja na prijatelje klase,
- Deklaracije prijatelja klase nisu obostrane (ako je A prijatelj od B, iz toga ne proizlazi da je B prijatelj od A),

- "Prijateljstvo" se ne nasljeđuje, odnosno, klase izvedene iz klase prijatelja nemaju automatski dozvoljen pristup privatnim i zaštićenim članovima osnovne klase. Također, klasa prijatelj nema automatski dozvoljen pristup privatnim i zaštićenim članovima klase koja je izvedena iz osnovne klase.

### 5.8.7 Nasljeđivanje

Nasljeđivanje je postupak kojim se iz postojeće klase izvodi nova, tako da ta nova klasa zadržava sve funkcijske i podatkovne članove osnovne klase, ali se njihova implementacija može promijeniti (vidi : Polimorfizam). Također nova klasa može uključivati i druge članove, osim naslijeđenih. Također, moguće je da jedna klasa naslijedi više osnovnih klasa (vidi: Višestruko nasljeđivanje). Pristup članovima osnovne klase iz objekata izvedene klase definiran je odgovarajućim specifikatorom tipa nasljeđivanja (private, protected ili public).

**Primjer (jednostavno nasljeđivanje) :**



```
#include <iostream.h>
```

```
class cvozilo{           // definira ono što je zajedničko za sva vozila
public:
```

```
    int godiste, tezina, snaga_motora, max_brzina;
    void ispis(){
        cout << "Vozilo : " << endl;
        cout << "Godiste : " << godiste << endl;
    };
};
```

```
class cautomobil : public cvozilo{    //klasa cautomobil nasljeđuje klasu cvozilo
};
```

```
class ckamion : public cvozilo{       //klasa ckamion nasljeđuje klasu cvozilo
public:
    int nosivost;
    void ispis(){                    // funkcijski član ispis je redefiniran
        cout << "Kamion : " << endl;
        cout << "Godiste : " << godiste << endl;
        cout << "Nosivost : " << nosivost << endl;
    };
};
```

```
class cautobus : public cvozilo{     //klasa cautobus nasljeđuje klasu cvozilo
public:
```

```

        int broj_sjedala;
};

cvozilo *vozilo; cautomobil *automobil; ckamion *kamion; cautobus *autobus;

void main (){
    vozilo=new cvozilo;
    automobil=new cautomobil;
    kamion=new ckamion;
    autobus=new cautobus;

    vozilo->godiste=1990;
    vozilo->ispis();

    automobil->godiste=1995;           // podatkovni član godiste je naslijeđen
                                     // od klase cvozilo,
    automobil->ispis();               // kao i funkcijski član ispis.

    kamion->godiste=1996;
    kamion->nosivost=5000;           // podatkovni član nosivost specifičan je
// za klasu ckamion
    kamion->ispis();                 // funkcijski član ispis je redefiniran u
//klasi ckamion
}

```

Ispisuje se slijedeće :

```

Vozilo :
Godiste : 1990
Vozilo :
Godiste : 1995
Kamion :
Godiste : 1996
Nosivost : 5000

```

#### 5.8.7.1 Tipovi nasljeđivanja

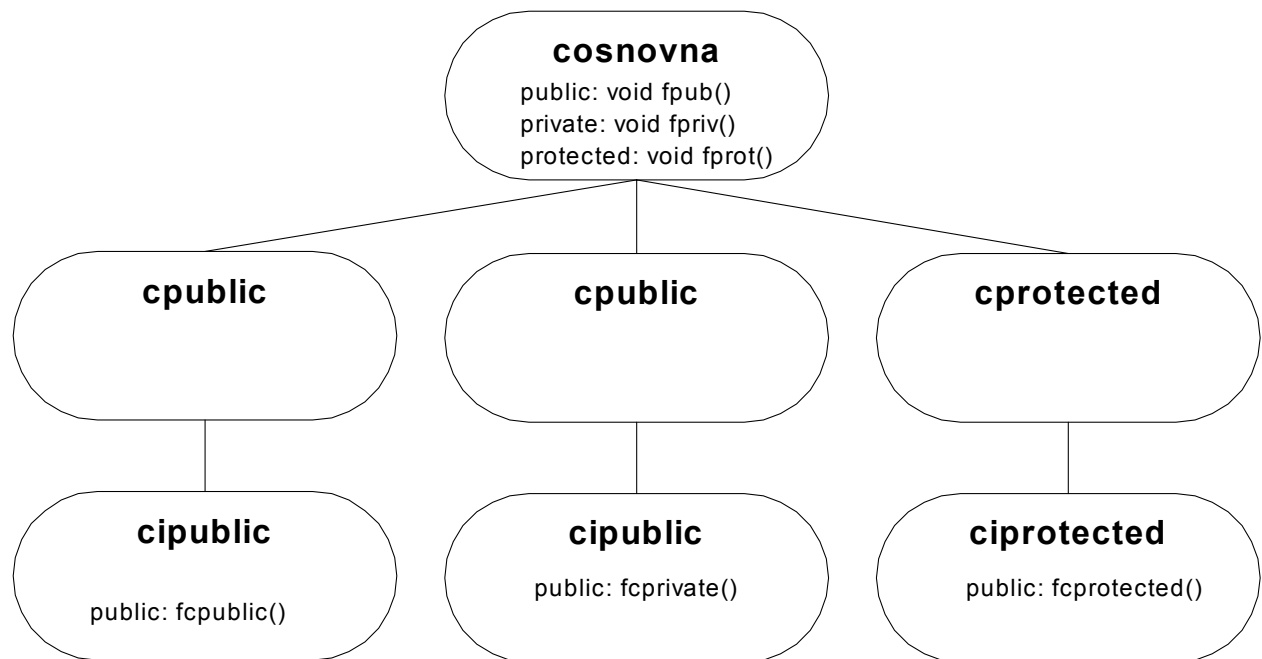
Kao što pojedini članovi klasa mogu biti javni, privatni ili zaštićeni, tako razlikujemo i odgovarajuće tipove nasljeđivanja (specifikator tipa nasljeđivanja navodi se u deklaraciji izvedene klase ispred naziva osnovne klase) :

- **Javno nasljeđivanje** (public) određuje da će svi podatkovni i funkcijski članovi osnovne klase na isti način biti dostupni iz izvedene klase kao i iz osnovne klase.

- **Privatno nasljeđivanje** (private) određuje da će članovi osnovne klase (osim privatnih) biti dostupni iz izvedene klase, ali se neće moći dalje nasljeđivati iz izvedene klase.

- **Zaštićeno nasljeđivanje** (protected) određuje da će članovi osnovne klase (osim privatnih) biti dostupni iz svih izvedenih klasa, ali ne i u ostalim dijelovima programa.

**Primjer (tipovi nasljeđivanja):**



```
#include <iostream.h>
```

```
class cosnovna{
public:
    void fpub(){
        cout << "Javni funkcijski clan!" << endl;
    };
private:
    void fpriv(){
        cout << "Privatni funkcijski clan!" << endl;
    };
protected:
    void fprot(){
        cout << "Zasticeni funkcijski clan!" << endl;
    };
};
```

```
class cpublic : public cosnovna{};
```

```
class cpublic : public cpublic{
public:
    void fcpublic(){
        cout << "Poziv naslijeđenih funkcijskih clanova kod javnog nasljeđivanja:" <<
            endl;
        fpub();
        // fpriv(); // Ovo, naravno, nije moguće jer se radi o privatnom članu!
        fprot();
    };
};
```

```
class cprivate : private cosnovna{};
```

```
class ciprivate : private cprivate{
```

```

public:
    void fcprivate(){
        cout << "Kod privatnog nasljeđivanja ništa nije ostalo 'unucima!'" << endl;
        // fpub(); fpriv(); fprot();
    };
};

class cprotected : protected cosnovna{};

class ciprotected : protected cprotected{
public:
    void fcprotected(){
        cout << "Zasticeno nasljedjivanje, kod poziva " << endl;
        cout << "iz izvedene klase nema razlike u " << endl;
        cout << "odnosu na javno nasljedjivanje..." << endl;
        fpub();
        //fpriv();
        fprot();
    };
};

cipublic javni; ciprivate privatni; ciprotected zasticeni;

void main(){
    javni.fcpublic();
    privatni.fcprivate();
    zasticeni.fcprotected();
    cout << "Ali kod poziva iz ostalih dijelova programa ima!" << endl;
    javni.fpub();
    //zasticeni.fpub();
}

```

Ispisuje se slijedeće:

```

Poziv nasljedjenih funkcijskih clanova kod javnog nasljedjivanja:
Javni funkcijski clan!
Zasticeni funkcijski clan!
Kod privatnog nasljedjivanja nista nije ostalo 'unucima!'
Zasticeno nasljedjivanje, kod poziva
iz izvedene klase nema razlike u
odnosu na javno nasljedjivanje...
Javni funkcijski clan!
Zasticeni funkcijski clan!
Ali kod poziva iz ostalih dijelova programa ima!
Javni funkcijski clan!

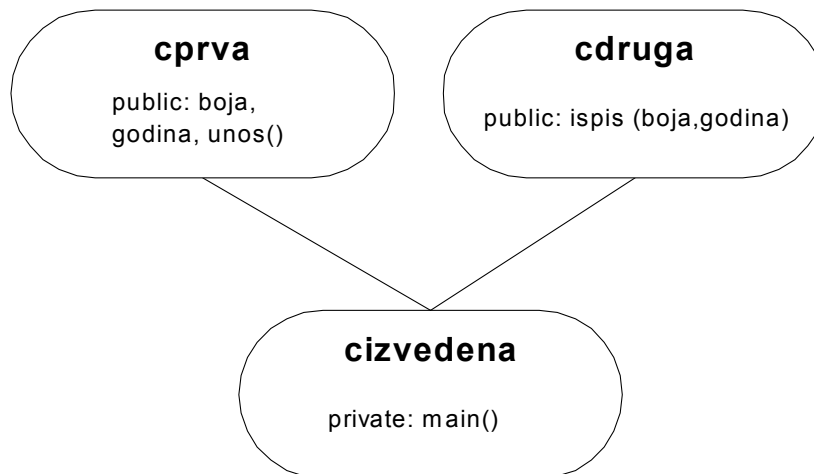
```

U slučaju da se ne navede eksplicitno tip nasljeđivanja, podrazumijeva se privatno nasljeđivanje.

### 5.8.7.2 Višestruko nasljeđivanje

Višestruko nasljeđivanje je takvo nasljeđivanje kod kojeg se klasa izvodi iz više osnovnih klasa. Na taj način moguće je kombinirati više razreda u jedan.

**Primjer (višestruko nasljeđivanje):**



```
#include <iostream.h>
```

```
class cprva{
public:
```

```
    int boja, godina;
    void unos(){
```

```
        cout << "Boja : ";
```

```
        cin >> boja;
```

```
        cout << "Godina : ";
```

```
        cin >> godina;
```

```
    };
```

```
};
```

```
class cdruga{
```

```
public:
```

```
    void ispis(int boja, int godina){
```

```
        cout << "Boja : " << boja << endl;
```

```
        cout << "Godina : " << godina << endl;
```

```
    };
```

```
};
```

```
class cizvedena : public cprva, public cdruga{;
```

```
// klasa cizvedena nasljeđuje sve članove klasa cprva i cdruga
```

```
cizvedena iobjekt;
```

```
void main(){
```

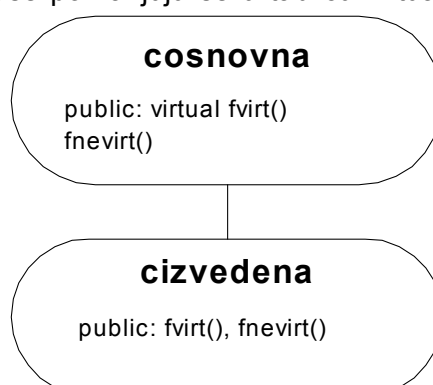
```
    iobjekt.unos();
```

```
    iobjekt.ispis(iobjekt.boja, iobjekt.godina);
```

```
}
```

## 5.8.8 Virtualni funkcijski članovi i polimorfizam

Funkcijski član klase može se deklarirati kao virtualan pomoću ključne riječi *virtual*. Svi virtualni funkcijski članovi klase pohranjuju se u tablicu virtualnih članova, a svaki objekt iz



osnovne klase (ali i izvedenih klasa, što je posebno bitno jer se time postiže *polimorfizam*) sadrži skriveni pokazivač na tu tablicu.

### Primjer (razlika između virtualnog i nevirtualnog funkcijskog člana):

```
#include <iostream.h>

class cosnovna{
public:
    virtual void fvirt(){
        cout << "Sada sam u funkciji cosnovna::fvirt"<<endl;}
    void fnevirt(){
        cout << "Sada sam u funkciji cosnovna::nevirt"<<endl;}
};

class cizvedena : public cosnovna{
public:
    void fvirt(){
        cout << "Sada sam u funkciji cizvedena::fvirt"<<endl;}
    void fnevirt(){
        cout << "Sada sam u funkciji cizvedena::nevirt"<<endl;}
};

void main () {
    cosnovna *objekt = new cizvedena;

    objekt -> fvirt();      // u prvom slučaju poziva se funkcijski član izvedene klase
    objekt -> fnevirt();    // a u drugom funkcijski član osnovne klase.
}
```

Ispisuje se slijedeće :

```
Sada sam u funkciji cizvedena::fvirt
Sada sam u funkciji cosnovna::nevirt
```

Smisao virtualnih funkcijskih članova je ostvarenje polimorfizma, na način da se objekti izvedenih klasa promatraju i kao objekti osnovnih klasa. U tom slučaju postoje različite implementacije istoimenih funkcijskih članova različitih izvedenih klasa.

Slijedeći primjer demonstrira implementaciju dvostruko vezane liste s heterogenim elementima. Pritom osnovna klasa (cvozilo) definira pokazivače na susjedne elemente liste, te virtualne funkcijske članove za unos i ispis podataka. Svaka od izvedenih klasa (cautomobil, ckamion i cautobus) definira svoje specifične podatkovne članove, te redefinira virtualne funkcijske članove. Time je omogućeno da se elementi liste definiraju iz osnovne klase cvozilo, ali se inicijaliziraju iz izvedenih klasa.

#### 5.8.8.1 Primjer (dvostruko vezana lista s heterogenim elementima)

```
#include <iostream.h>

class cvozilo{          // definira ono što je zajedničko za sva vozila
public:
    cvozilo *prethodni,*slijedeci;
```

```

    int godiste, max_brzina;
    virtual void unos(){};
    virtual void ispis(){};
};

class cautomobil : public cvozilo{    //klasa cautomobil nasljeđuje klasu cvozilo
public:
    void unos(){
        cout << "Automobil : " << endl;
        cout << "Godiste : ";
        cin >> godiste;
        cout << "Max. brzina : ";
        cin >> max_brzina;
    };
    void ispis(){
        cout << "Automobil : " << endl;
        cout << "Godiste : " << godiste << endl;
        cout << "Max. brzina : " << max_brzina << endl;
    };
};

class ckamion : public cvozilo{          //klasa ckamion nasljeđuje klasu cvozilo
public:
    int nosivost;
    void unos(){
        cout << "Kamion : " << endl;
        cout << "Godiste : ";
        cin >> godiste;
        cout << "Max. brzina : ";
        cin >> max_brzina;
        cout << "Nosivost : ";
        cin >> nosivost;
    };
    void ispis(){                          // funkcijski član ispis je redefiniran
        cout << "Kamion : " << endl;
        cout << "Godiste : " << godiste << endl;
        cout << "Max. brzina : " << max_brzina << endl;
        cout << "Nosivost : " << nosivost << endl;
    };
};

class cautobus : public cvozilo{    //klasa cautobus nasljeđuje klasu cvozilo
public:
    int broj_sjedala;
    void unos(){
        cout << "Autobus : " << endl;
        cout << "Godiste : ";
        cin >> godiste;
        cout << "Max. brzina : ";
        cin >> max_brzina;
        cout << "Broj sjedala : ";
        cin >> broj_sjedala;
    };
    void ispis(){
        cout << "Autobus : " << endl;

```



```

        cout << "Godiste : " << godiste << endl;
        cout << "Max. brzina : " << max_brzina << endl;
    };
};

cvozilo *lista;

void dodavanje(){           // dodavanje novog elementa liste i unos podataka
    int unos;
    cout << "Unesite 1,2 ili 3 (1. automobil 2. kamion 3. autobus) :";
    cin >> unos;
    cvozilo *zadnji,*novi;
    zadnji=lista;
    while (zadnji->sljedeći!=0)    // traženje zadnjeg elementa u listi
        zadnji=zadnji->sljedeći;
    switch (unos){
        case 1: novi = new cautomobil;break;
        case 2: novi = new ckamion;break;
        case 3: novi = new cautobus;break;
    };
    novi -> sljedeći = 0;
    novi -> prethodni = zadnji;
    zadnji -> sljedeći = novi;
    novi -> unos();
};

void ispisivanje(){         // ispis svih elemenata liste
    cvozilo *tekuci=lista->sljedeći;
    while (tekuci!=0){
        tekuci -> ispis();
        tekuci=tekuci->sljedeći;
    };
};

void main (){
    lista = new cvozilo;           // Kreiranje prvog elementa liste,
    lista -> prethodni=0;           // odnosno njenog zaglavlja
    lista -> sljedeći=0;
    int unos=0;
    do{
        cout << endl << "Hocete li:" << endl;
        cout << endl << "1. Dodavanje novog elementa liste." << endl;
        cout << "2. Ispis svih elemenata liste." << endl;
        cout << "9. Kraj programa." << endl << endl;
        cout << ">";
        cin >> unos;
        switch (unos){
            case 1: dodavanje();break;
            case 2: ispisivanje();
        };
    }while (unos != 9);
    cout << "Kraj programa." << endl;
}

```

Ispisuje se sljedeće :

Hocete li:

1. Dodavanje novog elementa liste.
2. Ispis svih elemenata liste.
9. Kraj programa.

>1

Unesite 1,2 ili 3 (1. automobil 2. kamion 3. autobus) :2

Kamion :

Godiste : 1994

Max. brzina : 100

Nosivost : 5000

odnosno :

Hocete li:

1. Dodavanje novog elementa liste.
2. Ispis svih elemenata liste.
9. Kraj programa.

>2

Kamion :

Godiste : 1994

Max. brzina : 100

Nosivost : 5000

Automobil :

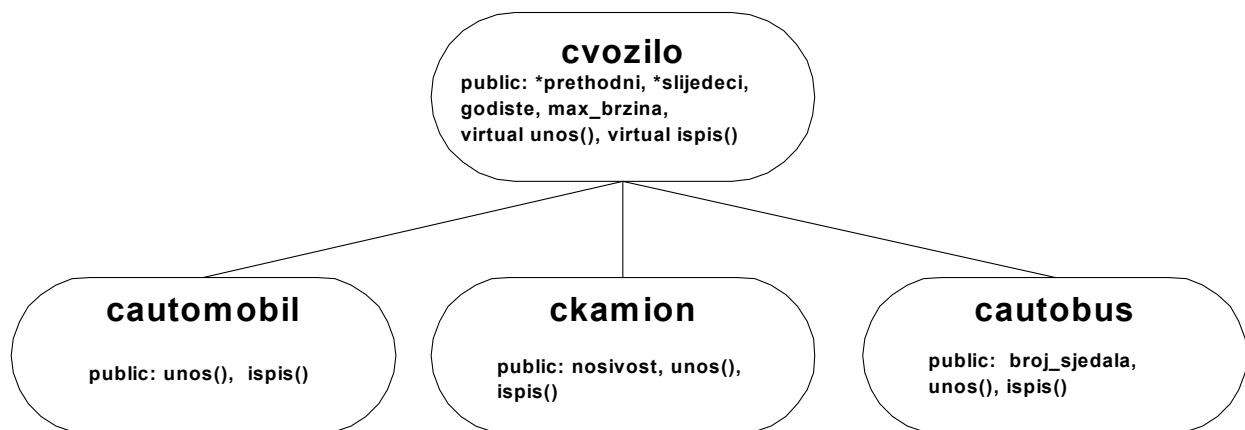
Godiste : 1990

Max. brzina : 140

Autobus :

Godiste : 1985

Max. brzina : 120



### 5.8.9 Apstraktne klase i čiste virtualne funkcije

Apstraktna klasa je klasa koja se koristi isključivo kao osnovna klasa za nasljeđivanje. Da bi klasa bila apstraktna, mora sadržavati najmanje jednu *čistu virtualnu funkciju*. Čista virtualna funkcija je virtualni funkcijski član iza čije deklaracije stoji oznaka `=0`. Nije moguće deklarirati

objekte apstraktnih klasa, ali se zato mogu deklarirati pokazivači i reference na te klase. Čiste virtualne funkcije čine javno sučelje koje se detaljno specificira u izvedenim klasama.

**Primjer (apstraktna klasa) :**



```
include <iostream.h>
class capstraktna{
public:
    int boja,godina;
    virtual void unos()=0;
    virtual void ispis()=0;
};
class cizvedena : public capstraktna{
public:
    void unos(){                // definicija funkcija koje su u osnovnoj
        cout << "Boja : ";      // klase definirane kao čiste virtualne
        cin >> boja;
        cout << "Godina : ";
        cin >> godina;
    };
    void ispis(){
        cout << "Boja : " << boja << endl;
        cout << "Godina : " << godina << endl;
    };
};
//capstraktna objekt; // ne može se definirati objekt iz apstraktne klase
cizvedena iobjekt;
void main(){
    iobjekt.unos();
    iobjekt.ispis();
}
```

Napomena : ako se u izvedenom razredu ne definiraju sve čiste virtualne funkcije osnovne klase, izvedena klasa će također biti apstraktna.